

Multi-user Messenger Program

Computer Networks Design Project Report

Project Goal

Design and implement a simple text-based messenger where a login server manages online user information, while clients exchange chat messages directly using an HTTP-like protocol.

Course	Computer Networks
Project Type	Design and implementation
Team Name	Temuujin
Implementation Language	Python 3
User Interface	Text-based terminal UI
Main Protocol Style	HTTP-like start line + headers + blank line + body

Summary

This project implements the requested messenger using one login server and multiple user client programs. The login server only maintains online user discovery information: user ID, IP address, and port number. Actual messages are transmitted directly from one client to another, so the architecture matches the peer-to-peer behavior required by the project specification.

1. Requirement Coverage

The table below maps each project requirement to the implemented feature. This is the important part, because professors enjoy requirement checklists almost as much as students enjoy debugging ports at midnight.

Professor requirement	Implementation	Where
Login server stores currently online users	Keeps a synchronized dictionary of online users and writes it to JSON.	login_server.py
Store user ID, IP address, and port number	Each REGISTER request contains User-ID, IP, and Listen-Port headers.	client.py + login_server.py
New client receives other online users	After registration, server returns a JSON list excluding the registering user.	REGISTER /login
Show online users to user	The users command prints a table with ID, IP address, and port.	client.py
Messages do not pass through login server	Clients connect directly to other clients using stored IP/port.	send_to_peer()
Invite users to messenger session	invite <user_id> sends INVITE /session/control directly to the selected peer.	client.py
End messenger session	end sends END messages and clears local session state.	client.py
Send message to all users in session	send <message> iterates over the current session set and sends MSG to each peer.	client.py
HTTP-like message format	Every protocol message has a start line, headers, blank line, and body with Content-Length.	protocol.py

Design Scope

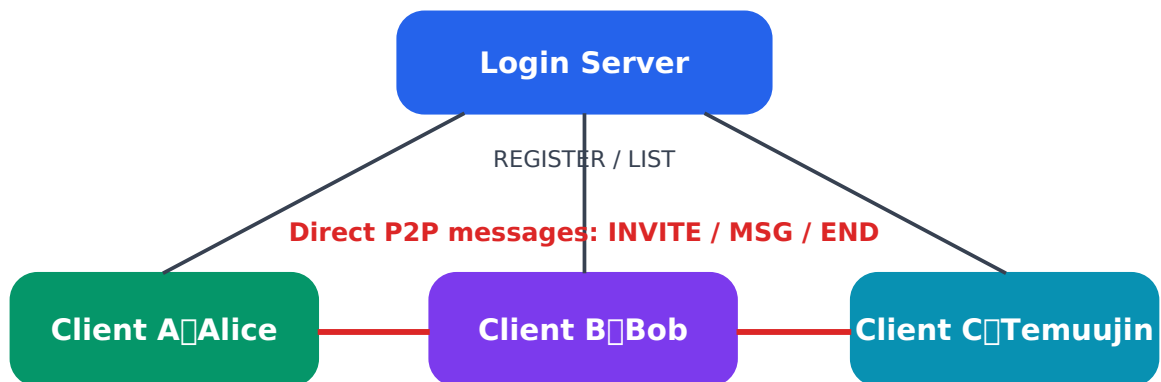
The implementation intentionally stays simple: a terminal interface, socket-based communication, JSON storage for the online-user table, and thread-based handling for concurrent peer/server connections. This keeps the project within the requested small design size while still covering all mandatory functions.

Important Fix

The login server clears old online users by default when it starts. This prevents stale users from appearing in the online list before their client programs are actually running.

2. System Architecture

The system has two types of programs: a login server and user clients. Each client also acts as a small server because it listens for direct peer messages from other clients.



The login server is not used to forward chat messages.

Login Server Responsibilities

The login server accepts REGISTER, LIST, and UNREGISTER requests. It stores the currently online users in a JSON file. Each row contains the user ID, IP address, and listening port. It does not participate in chat message forwarding.

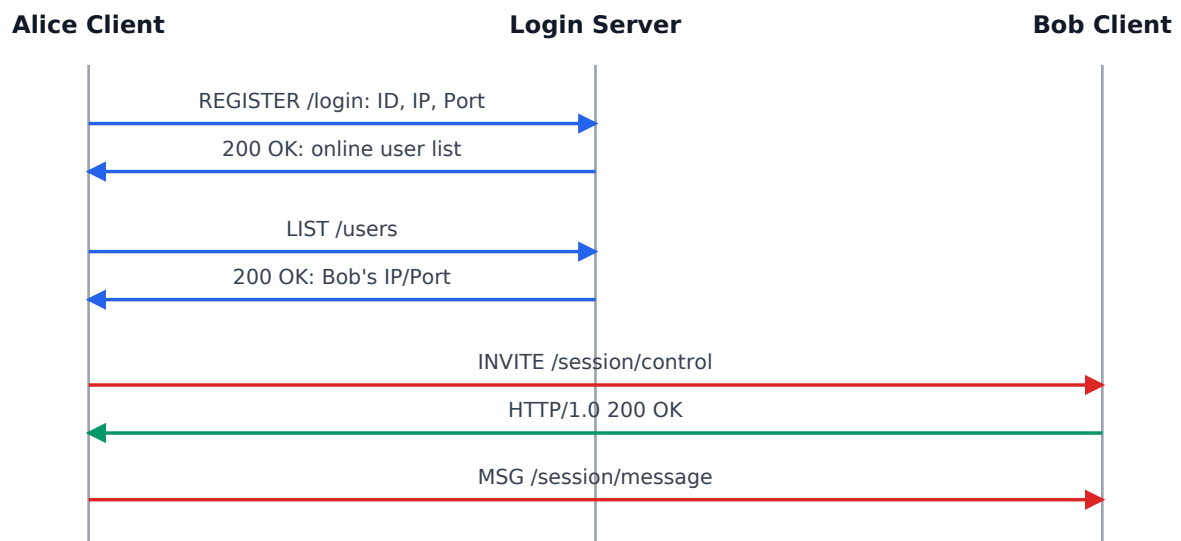
Client Responsibilities

Each client registers itself with the login server, receives the online user list, displays available users, accepts commands from the terminal, listens for incoming peer requests, and sends direct HTTP-like messages to invited session members.

File	Responsibility
protocol.py	Builds and parses HTTP-like request/response messages. Handles Content-Length and body reading.
login_server.py	Registers clients, stores online user table, returns available users, unregisters users on exit.
client.py	Runs the text UI, peer listener, direct peer messaging, session invite/end/send commands.
online_users.json	Generated at runtime. Stores current online users as ID/IP/port records.

3. Communication Flow

A normal run follows this sequence. The login server is used only for discovery. Once Alice knows Bob's IP address and port number, Alice sends INVITE and MSG requests directly to Bob.



Protocol Methods

Method	Path	Purpose
REGISTER	/login	Client sends its ID/IP/port to the login server and receives the online user list.
LIST	/users	Client refreshes the current online user list from the login server.
UNREGISTER	/logout	Client asks the login server to remove it from the online list.
INVITE	/session/control	Client invites another online user directly into the current messenger session.
MSG	/session/message	Client sends a chat message directly to each current session member.
END	/session/control	Client ends the session and notifies previous session members.

4. HTTP-like Message Format

Every message follows the same basic structure: a start line, several headers, an empty line, and an optional body. This matches the project requirement that the message format should be extensible like HTTP Request/Response messages.

Example: Client registration

```
REGISTER /login HTTP/1.0
User-ID: alice
IP: 127.0.0.1
Listen-Port: 5001
Content-Length: 0
```

Example: Server response with online users

```
HTTP/1.0 200 OK
Content-Type: application/json
Content-Length: 62
```

```
[
  {"id": "bob", "ip": "127.0.0.1", "port": 5002}
]
```

Example: Direct peer-to-peer chat message

```
MSG /session/message HTTP/1.0
From: alice
To: bob
Session-Mode: direct-peer-to-peer
Content-Length: 30
```

```
hello bob, this thing works
```

Extensibility

Because the format uses headers, additional features can be added later without changing the entire protocol. For example, Session-ID, Timestamp, Message-ID, or Content-Type headers could be inserted in the future.

5. Terminal UI and User Commands

When a user joins, the client prints a welcome screen explaining what the program does and how to terminate the process safely. The UI separates online users from current session members, which prevents confusion between users who are merely online and users who were actually invited.

Welcome Screen

Welcome, alice
This is a simple peer-to-peer messenger for the network project. The login server stores only online users: ID, IP address, and port. Actual chat messages are sent directly between clients. Type 'help' to show commands again. Type 'exit' or 'quit' to terminate this process safely.

Commands

Command	Meaning
users	Refresh and print online users from the login server.
invite <user_id>	Invite a selected online user into the current messenger session.
session	Show current session members.
send <message>	Send one HTTP-like MSG request to every user in the current session.
end	End the session and notify session members.
help	Print command guide again.
exit / quit	End the session, unregister from login server, and terminate the process.

6. How to Run and Test

The project can be demonstrated on one computer using localhost and three terminals. This satisfies the network behavior because the programs still communicate through TCP sockets and port numbers.

Step 1: Start login server

```
python3 login_server.py --host 127.0.0.1 --port 9000
```

Step 2: Start Alice

```
python3 client.py --id alice --listen-port 5001 --server-port 9000
```

Step 3: Start Bob

```
python3 client.py --id bob --listen-port 5002 --server-port 9000
```

Step 4: Test session behavior

```
users
invite bob
session
send hello bob, this messenger works
end
exit
```

Expected Result

Bob appears as an online user only after Bob's client is running. Bob is not part of Alice's current session until Alice executes `invite bob`. After `send`, Bob receives the direct message and Alice prints the outgoing HTTP-like packet.

Demo Tip

During the video demo, show three terminals: login server, Alice, and Bob. Run `users`, `invite`, `send`, `end`, and `exit`. Keep `--show-packets` enabled if you want to show `INVITE` and `END` packets too.

7. Implementation Notes

Concurrency

The login server uses threads so multiple clients can register or request the online-user list. Each client also starts a peer listener thread so it can receive messages while the user is typing commands.

State Management

The login server state is the online-user dictionary. The client state has two separate parts: `online_users` and `session`. This is important because online users are only candidates to invite, while session members are the actual message targets.

Storage

The login server writes the online-user dictionary to `online_users.json`. On startup, the fixed version clears stale entries by default because old clients may have terminated without unregistering cleanly.

Limitations

This implementation does not include passwords, encryption, persistent chat history, or a graphical interface. Those features are intentionally omitted because the assignment requires a simple messenger focused on networking architecture and message format.

Conclusion

The final design satisfies the required functions: online-user management through a login server, direct client-to-client messaging, inviting users, ending sessions, broadcasting to current session members, and using an HTTP-like extensible message format. The UI remains simple but clean enough for a demo without looking like it was assembled in a cave with a dying keyboard.